SQL 与 MYSQL 基础知识 + 问答手册

这是一份面向面试复习的 SQL / MySQL 手册，重点覆盖：

- 基础概念
- 常用语法
- 高频问答
- 经典 SQL 模板
- 索引、事务、锁、MVCC、优化
- 面试速记

说明：

- 本手册以 MySQL 为主，兼顾通用 SQL 知识。
- 你提供的知乎链接由于访问限制未能稳定抓取正文，因此这里整理的是一份完整的通用面试手册，而不是对原文逐段转写。
- 如果后续你贴出原文内容，可以继续补成“原题对应版”。

---

1. SQL 和 MYSQL 是什么

1.1 什么是 SQL

SQL，Structured Query Language，结构化查询语言，是用来操作关系型数据库的标准语言。

从本质上说，SQL 不是某一个数据库厂商独有的语言，而是一套围绕“关系模型”设计出来的表达方式。它让我们能够用接近自然语言的形式描述：

- 我要查哪张表
- 我只关心哪些字段
- 我要满足什么条件的数据
- 数据要怎么分组、排序、统计
- 数据该如何新增、修改、删除

SQL 的核心思想不是“怎么一步步执行”，而是“你想得到什么结果”。这也是它和很多编程语言不一样的地方。你写 SQL 时更像是在描述目标，至于具体怎么扫描、怎么选索引、怎么排序，通常由数据库优化器决定。

它可以完成：

- 定义数据库和表
- 插入、删除、修改数据
- 查询数据
- 管理权限
- 控制事务

1.2 什么是 MySQL

MySQL 是常见的关系型数据库管理系统，使用表的形式存储数据，广泛用于互联网业务系统。

更准确地说，MySQL 做了三件事：

- 帮你把数据安全地存到磁盘上
- 帮你高效地把数据查出来
- 在多人同时读写时尽量保证数据正确

所以 MySQL 不只是“存数据的地方”，它还是一个同时负责存储、查询、并发控制、故障恢复的系统。

面试里说 MySQL 时，通常不只是在说“数据库软件”，还隐含包含这些能力：

- 存储引擎，如 InnoDB
- 索引结构，如 B+Tree
- 事务与锁
- 日志与恢复
- 优化器和执行计划

简单理解：

- 数据库是存数据的仓库
- 表是仓库里的一个个货架
- 行是一条记录
- 列是一个字段

1.3 常见数据库术语

- `database`：数据库，是一组表、视图、索引等对象的集合
- `table`：数据表，是存储数据的基本单位
- `row`：行，记录，表示一个具体对象的一条数据
- `column`：列，字段，表示某个属性
- `primary key`：主键，用来唯一标识一行数据
- `foreign key`：外键，用来描述表与表之间的关联关系
- `index`：索引，是帮助数据库加速查找的数据结构
- `view`：视图，是基于 SQL 查询结果构造的逻辑表
- `transaction`：事务，是一组要么都成功、要么都失败的操作

1.4 什么是关系型数据库

关系型数据库的核心不是“表长什么样”，而是它背后的关系模型。

你可以这样理解：

- 现实世界里有对象，如用户、订单、商品
- 关系型数据库把这些对象拆成一张张表
- 每张表通过字段来描述对象属性
- 表和表之间通过主键、外键或业务字段建立联系

比如：

- `user` 表存用户
- `orders` 表存订单
- `orders.user_id` 指向 `user.id`

这样数据库就能表达“哪个订单属于哪个用户”。

关系型数据库的优点：

- 结构清晰
- 数据一致性较强
- 查询能力强
- 适合复杂关联分析

1.5 为什么业务系统常用关系型数据库

因为业务系统里经常有这些需求：

- 下单时要扣库存
- 支付时要修改订单状态
- 用户、订单、商品之间要做关联查询

- 多个人同时操作时要保证数据不能乱

这些场景都非常依赖：

- 结构化数据
- 多表关联
- 事务能力
- 一致性约束

而这正是 MySQL 这类关系型数据库最擅长的地方。

---

2. SQL 分类

2.1 DDL

DDL，Data Definition Language，数据定义语言，用来定义数据库对象。

所谓“定义数据库对象”，本质上是在告诉数据库：

- 这个表叫什么名字
- 表里有哪些字段
- 每个字段是什么类型
- 哪些字段不能为空
- 哪些字段必须唯一
- 哪些字段要建索引

所以 DDL 操作的不是“数据内容”，而是“数据长什么样”。

常见语句：

- `create`

- `alter`
- `drop`
- `truncate`

2.2 DML

DML，Data Manipulation Language，数据操作语言，用来增删改数据。

它操作的是表中的实际数据记录，也就是行数据。

比如：

- 新增一个用户
- 修改订单状态
- 删除无效记录

这些都是 DML。

常见语句：

- `insert`
- `update`
- `delete`

2.3 DQL

DQL，Data Query Language，数据查询语言。

它最核心的任务是从已有数据中提取信息。
严格来说，业务系统里绝大多数"读操作"都属于 DQL。

查询并不只是"把数据拿出来"，它还包括：

- 按条件过滤
- 多表关联

- 聚合统计
- 排序分页
- 排名分析

常见语句：

- `select`

2.4 DCL

DCL，Data Control Language，数据控制语言。

这类语句主要解决“谁能看、谁能改”的问题。

常见语句：

- `grant`
- `revoke`

2.5 TCL

TCL，Transaction Control Language，事务控制语言。

这类语句解决的是“多条 SQL 要不要看成一个整体执行”的问题。

例如转账场景中，扣钱和加钱必须同时成功，否则数据就会不一致。
这时就需要 TCL 来配合事务。

常见语句：

- `commit`
- `rollback`
- `savepoint`

---

3. MYSQL 常见数据类型

3.1 数值类型

- `tinyint`
- `smallint`
- `int`
- `bigint`
- `float`
- `double`
- `decimal`

面试重点：

- 金额建议使用 `decimal`
- `float` 和 `double` 存在精度问题

概念上可以这样理解：

- 整数类型用于存离散的整数值，如年龄、数量、状态
- 浮点类型用于近似小数
- 定点类型用于精确小数

为什么金额适合 `decimal`：

- 因为金额计算要求精确
- `float/double` 底层是二进制浮点表示
- 某些十进制小数无法精确表示
- 会造成精度误差

3.2 字符串类型

- `char(n)`：定长字符串
- `varchar(n)`：变长字符串
- `text`：长文本

面试重点：

- `char` 适合长度固定的数据，如身份证号、状态码
- `varchar` 更灵活，业务场景更常用

再往深一层理解：

- `char` 更像"固定格子"，不管实际是否占满，都按固定长度处理
- `varchar` 更像"可伸缩格子"，按真实长度存储

所以：

- `char` 读取和比较时比较直接
- `varchar` 更省空间
- 大部分业务字段长度并不固定，因此 `varchar` 更常见

3.3 日期时间类型

- `date`
- `time`
- `datetime`
- `timestamp`

面试重点：

- `datetime` 取值范围大
- `timestamp` 常用于记录更新时间，与时区关系更明显

日期时间类型的本质是帮助数据库表达“某件事发生在什么时候”。

常见区别可以这样理解：

- `date` 只关心日期，不关心时分秒
- `time` 只关心时间
- `datetime` 同时保存日期和时间
- `timestamp` 常用于系统时间戳、创建时间、更新时间

---

4. 建表与约束

4.1 常见约束

- `primary key`：主键，唯一且非空
- `unique`：唯一约束
- `not null`：非空
- `default`：默认值
- `foreign key`：外键

约束的本质，是让数据库帮我们维护数据规则，而不是完全依赖程序员自己小心。

例如：

- 用户 id 不能重复
- 用户名不能为空
- 订单必须属于某个用户

这些规则如果只写在代码里，很容易因为程序 bug 被破坏；而数据库约束能在更底层兜住一部分错误。

4.2 建表示例

```sql
create table user (
    id bigint primary key auto_increment,
    name varchar(50) not null,
    email varchar(100) unique,
    age int default 18,
    created_at datetime default current_timestamp
);
```

4.3 主键和唯一约束区别

- 主键不能为 `null`
- 一个表只能有一个主键
- 一个表可以有多个 `unique`
- 主键通常是表中最核心的唯一标识

还可以这样理解：

- 主键偏"身份标识"
- 唯一约束偏"业务唯一性"

例如用户表中：

- `id` 可以作为主键
- `email` 可以加唯一约束

前者表示"系统内部靠谁识别这条记录"，后者表示"业务上这个值不能重复"。

---

5. 基础 SQL 语法总览

最常见的查询结构：

```
select 字段列表
from 表名
where 条件
group by 分组字段
having 分组后条件
order by 排序字段
limit 分页;
```

理解这条语句最重要的一点是：
SQL 查询本质上是在做一条“数据加工流水线”。

你可以把它理解成：

1. 先决定从哪取数据
2. 再决定保留哪些行
3. 再决定是否分组
4. 再决定是否做聚合统计
5. 最后决定怎么展示结果

所以一个复杂查询，其实就是对“结果集”不断加工的过程。

5.1 SQL 逻辑执行顺序

SQL 的书写顺序不是执行顺序。逻辑执行顺序通常是：

1. `from`
2. `join`
3. `on`
4. `where`

5. `group by`
6. `having`
7. `select`
8. `distinct`
9. `order by`
10. `limit`

这是非常高频的面试题。

为什么这个顺序重要：

- 因为它能解释很多“为什么这么写会报错/结果不对”的问题
- 例如 `where` 里不能直接用聚合结果，因为聚合还没发生
- 例如 `having` 能过滤分组结果，因为它发生在 `group by` 之后

理解了执行顺序，很多 SQL 问题都更容易分析。

---

6. 基础查询语法

查询语句的本质，是从一张表或多张表中拿到一个新的结果集。

这个结果集不是必须真的存到磁盘里，它更多是“逻辑上算出来的一张临时表”。

所以你写 `select` 时，可以始终问自己两个问题：

1. 输入数据是什么
2. 输出结果长什么样

6.1 查询所有数据

```
select * from emp;
```

6.2 查询指定字段

```
select id, name, salary from emp;
```

6.3 别名

```
select name as employee_name, salary as emp_salary
from emp;
```

6.4 去重

```
select distinct dept_id from emp;
```

6.5 条件查询

```
select * from emp where salary > 10000;
select * from emp where age between 20 and 30;
select * from emp where dept_id in (1, 2, 3);
select * from emp where name like '张%';
```

6.6 模糊查询

- `%` 表示任意多个字符
- `_` 表示任意一个字符

```
select * from user where name like 'a%';
select * from user where name like '_b%';
```

6.7 排序

```
select * from emp order by salary desc, id asc;
```

6.8 分页

```
select * from emp limit 10;
select * from emp limit 0, 10;
select * from emp limit 20, 10;
```

分页的本质是“只取结果集中的一小段”。
在前端列表页、管理后台、订单记录查询里都非常常见。

但要注意：

- SQL 分页并不是数据库天然“只看这几条”
- 很多时候数据库还是要先扫描前面的数据，再截取一段返回
- 所以页数很深时性能会变差

---

7. 常用聚合函数

聚合函数的本质，是把“多行数据压缩成一个统计结果”。

例如：

- 多行工资压缩成平均工资
- 多行订单压缩成总金额
- 多行用户压缩成人数

所以聚合的核心思想是“从明细变成统计”。

常见聚合函数：

- `count()`
- `sum()`
- `avg()`
- `max()`
- `min()`

示例：

```
select count(*) from emp;

select dept_id, avg(salary) as avg_salary
from emp
group by dept_id;
```

7.1 count 的区别

- `count(*)`：统计总行数
- `count(1)`：通常也统计总行数
- `count(字段)`：统计该字段不为 `null` 的行数

面试里如果问你"为什么 `count(字段)` 和 `count(*)` 不一样"，关键点就在这里：

- `count(*)` 关心的是行
- `count(字段)` 关心的是该字段有没有值

---

8. GROUP BY 和 HAVING

分组的本质，是把"多行数据按某个维度归类"。

比如按部门分组、按城市分组、按月份分组。
一旦分组以后，你关注的就不再是某一行明细，而是"这一组整体"。

8.1 group by

用于分组统计。

```
select dept_id, count(*) as cnt
from emp
group by dept_id;
```

8.2 where 和 having 的区别

- `where`：分组前过滤
- `having`：分组后过滤

更直观地理解：

- `where` 决定"哪些原始行可以进入统计"
- `having` 决定"哪些统计结果可以保留下来"

示例：

```
select dept_id, count(*) as cnt
from emp
group by dept_id
having count(*) > 5;
```

---

9. 多表查询

多表查询的本质，是把分散在不同表中的信息重新拼起来。

现实业务里，一张表往往放不下所有信息：

- 用户信息在用户表
- 订单信息在订单表

- 商品信息在商品表

如果你想看“哪个用户买了什么商品”，就必须做多表查询。

假设两张表：

- `student(id, name, class_id)`
- `class(id, class_name)`

9.1 内连接

```
select s.name, c.class_name
from student s
inner join class c on s.class_id = c.id;
```

含义：

- 只保留左右表匹配成功的数据

9.2 左连接

```
select s.name, c.class_name
from student s
left join class c on s.class_id = c.id;
```

含义：

- 保留左表全部数据
- 右表匹配不上时补 `null`

9.3 右连接

```
select s.name, c.class_name
from student s
right join class c on s.class_id = c.id;
```

9.4 自连接

```sql
select e.name, m.name as manager_name
from emp e
left join emp m on e.manager_id = m.id;
```

9.5 on 和 where 的区别

- `on` 是连接条件
- `where` 是结果过滤条件

可以这样区分：

- `on` 决定两张表怎么拼
- `where` 决定拼完之后保留哪些结果

注意：

左连接中，如果把右表过滤条件写进 `where`，可能导致结果接近内连接。

```sql
select *
from a
left join b on a.id = b.a_id
where b.status = 1;
```

---

10. 子查询

子查询的本质，是“把一个查询的结果，作为另一个查询的输入”。

它适合用在这种场景：

- 先查出平均值，再和明细比较
- 先查出一批 id，再去主表中过滤

- 先做一轮统计，再对统计结果继续筛选

10.1 标量子查询

子查询返回单个值。

```
select *
from emp
where salary > (select avg(salary) from emp);
```

10.2 in 子查询

```
select *
from emp
where dept_id in (
    select id from dept where name = '技术部'
);
```

10.3 from 子查询

```
select *
from (
    select dept_id, avg(salary) as avg_salary
    from emp
    group by dept_id
) t
where t.avg_salary > 10000;
```

10.4 exists 子查询

```
select *
from a
where exists (
    select 1 from b where b.a_id = a.id
);
```

10.5 in 和 exists 的常见理解

- `in` 常用于子查询结果较小
- `exists` 常用于判断是否存在匹配记录

面试中可以这样答，但实际执行还要看优化器。

所以你在面试里最好补一句：

- 这是经验性结论
- 最终还是要结合执行计划判断

---

11. UNION 和 UNION ALL

这一类操作的本质，是把两个结果集合并起来。

它和 `join` 的区别要搞清楚：

- `join` 是横向拼表
- `union` 是纵向拼结果集

11.1 union

合并结果并去重。

```
select name from t1
union
select name from t2;
```

11.2 union all

合并结果但不去重。

```
select name from t1
union all
select name from t2;
```

面试重点：

- `union all` 通常比 `union` 快
- 因为 `union` 需要额外去重

---

12. 常用函数

函数的本质是对字段值进行加工，让结果更符合业务展示或统计要求。

例如：

- 把姓名拼起来
- 把日期格式化成年月
- 把分数转换成等级
- 把空值替换成默认值

12.1 字符串函数

- `concat()`
- `length()`
- `substring()`
- `trim()`
- `upper()`

- `lower()`

```
select concat(first_name, last_name) from user;
```

12.2 数值函数

- `round()`
- `ceil()`
- `floor()`
- `abs()`

12.3 日期函数

- `now()`
- `curdate()`
- `date_add()`
- `datediff()`
- `date_format()`

```
select now();
select date_add(now(), interval 7 day);
```

12.4 条件函数

- `if()`
- `case when`

```sql
select name,
       case
           when score >= 90 then 'A'
           when score >= 80 then 'B'
           else 'C'
       end as level
from student;
```

---

13. 窗口函数

MySQL 8.0+ 支持窗口函数。

窗口函数是很多人第一次学 SQL 时最难理解的一类函数。

它和普通聚合函数最大的区别是：

- 普通聚合函数会"压缩行数"
- 窗口函数通常"不压缩行数"

也就是说，窗口函数能在保留每一行明细的同时，为每一行补充一个统计结果或排名结果。

这使它特别适合：

- 排名
- TopN
- 累计求和
- 同比环比
- 连续登录

常见函数：

- `row_number()`
- `rank()`

- `dense_rank()`
- `sum() over()`
- `avg() over()`

13.1 每个部门工资前 3 名

```
select *
from (
    select e.*,
           row_number() over(partition by dept_id order by
salary desc) as rn
    from emp e
) t
where rn <= 3;
```

13.2 row_number、rank、dense_rank 区别

- `row_number()`：连续编号，不考虑并列
- `rank()`：并列后跳号
- `dense_rank()`：并列后不跳号

---

14. NULL 的理解

`null` 表示未知，不等于 0，也不等于空字符串。

理解 `null` 时，千万不要把它理解成"一个特殊值"，更准确地说，它表示"这个值目前不存在或未知"。

例如：

- 用户还没填写邮箱
- 订单还没有支付时间

- 某个字段不适用于这条数据

这也是为什么 `null` 在 SQL 里有很多特殊规则，因为“未知值”和普通值做比较，本身就不容易得到确定结果。

14.1 正确判断 null

```
select * from user where email is null;
select * from user where email is not null;
```

不能写：

```
where email = null
```

14.2 面试重点

- `null` 参与计算，结果通常还是 `null`
- `count(字段)` 不统计 `null`
- MySQL 中唯一索引通常允许多个 `null`

你可以顺手记住一个核心思路：

- 遇到 `null`，要优先想“未知”而不是“空”

---

15. 索引

索引是面试最重要的模块之一。

15.1 什么是索引

索引本质上是一种帮助数据库快速查找数据的数据结构。

你可以把它类比成书的目录。

- 如果一本书没有目录，你想找“事务”这一章，就只能从第一页往后翻
- 如果有目录，你就可以先定位到对应页码，再快速找到内容

数据库也一样。

假设有一张用户表，里面有一百万条记录：

```
select * from user where id = 10001;
```

如果没有索引，数据库可能需要一行一行检查，直到找到这条记录，这叫全表扫描。
如果有索引，数据库就可以通过索引结构快速定位到目标数据。

所以索引并不是“保存数据本身的新表”，而是数据库为了提高查询效率，额外维护的一套查找结构。

15.2 为什么索引能加快查询

因为索引会提前按照某种规则把数据组织好，让“查找”不再靠从头扫描。

没有索引时，查找的思路通常是：

- 把所有记录挨个比对

有索引时，查找的思路通常是：

- 先通过索引缩小范围
- 再精确定位目标记录

它快的核心原因不是“数据库更聪明了”，而是“数据库少看了很多无关数据”。

面试里你可以这样答：

- 索引的本质是用空间换时间
- 通过额外存储和维护索引结构，换取查询速度提升

15.3 索引的代价是什么

索引不是越多越好，因为索引也有成本。

主要成本包括：

- 占用额外存储空间
- 插入、更新、删除时要同步维护索引
- 索引过多会增加写入开销
- 可能让优化器选择变复杂

例如：

- 表里插入一条数据时，不只是往数据页写一条记录
- 还可能要更新一个或多个索引结构

所以索引的本质不是“免费加速器”，而是一种需要权衡的设计。

15.4 什么场景需要索引

通常这些场景适合建索引：

- 高频查询的条件字段
- 多表关联时的连接字段
- 经常排序的字段
- 经常分组的字段
- 需要唯一约束的字段

例如：

- 用户表里的 `id`
- 订单表里的 `user_id`
- 按时间倒序查询的 `create_time`

15.5 什么场景不适合索引

这些场景通常不适合盲目建索引：

- 表数据量很小
- 字段重复值特别多，区分度很低
- 这个字段几乎不参与查询
- 写入特别频繁

例如“性别”字段通常只有男、女两个值，区分度很低，单独建索引的收益往往有限。

15.6 索引和主键是什么关系

很多初学者会把主键和索引混成一个概念，但它们不完全一样。

- 主键是一种约束，强调唯一标识
- 索引是一种数据结构，强调加速查找

在 InnoDB 中：

- 主键会天然对应一个主键索引
- 所以你经常会看到“主键索引”这个说法

但并不是说“只有主键才有索引”，普通字段、唯一字段、联合字段也都可以建索引。

15.7 索引是怎么建立的

这个问题要分成两个层面理解：

1. 作为开发者，我们是怎么“声明”一个索引的
2. 作为数据库，InnoDB 是怎么把这个索引真正建出来的

SQL 层面怎么创建索引

最常见的创建方式有几种。

在建表时直接定义主键或唯一约束：

```
create table user (
    id bigint primary key auto_increment,
    email varchar(100) unique,
    name varchar(50),
    age int
);
```

建表后单独创建普通索引：

```
create index idx_name on user(name);
```

创建联合索引：

```
create index idx_name_age on user(name, age);
```

通过 `alter table` 创建索引：

```
alter table user add index idx_city(city);
alter table user add unique index uk_phone(phone);
```

这部分可以理解成：
我们通过 SQL 告诉 MySQL，“请你为这些列额外维护一套高效查找结构”。

数据库内部是怎么建出索引的

如果从 InnoDB 内部看，建立索引并不是“打个标签”那么简单，而是要把索引列的值重新组织成一棵 B+Tree。

大致过程可以理解为：

1. 读取表中的已有记录

2. 取出被索引列的值
3. 按索引键值进行排序和组织
4. 构造成 B+Tree 的叶子节点和非叶子节点
5. 把这棵树保存到页中，形成真正的索引结构

如果是主键索引：

- 叶子节点存的是整行记录

如果是二级索引：

- 叶子节点存的是索引列值 + 主键值

所以“建立索引”的本质不是加一个标记，而是新建一套按键值组织的数据结构。

建好之后索引如何维护

索引不是建完就不管了。
后续每次 `insert`、`update`、`delete`，数据库都要同步维护对应的 B+Tree。

例如：

- 插入一条记录时，要把新的键插入索引树
- 删除一条记录时，要从索引树中删除对应键
- 更新索引列时，本质上可能是“删旧键 + 插新键”

这也是为什么索引越多，写入成本越高。

15.8 建立主键索引和普通索引有什么区别

主键索引的建立

在 InnoDB 中，主键索引不是“附属结构”，而是整张表数据的组织方式。

也就是说：

- 建立主键索引时，数据库会按主键顺序组织整行数据

- 主键索引的叶子节点中直接存放完整记录

所以主键索引一旦确定，整张表在物理组织上就有了核心顺序。

普通索引的建立

普通索引是建立在主键索引之外的辅助结构。

例如有表：

```
user(id, name, age)
```

如果你在 `name` 上建立普通索引：

```
create index idx_name on user(name);
```

那么 InnoDB 会额外维护一棵以 `name` 为键的 B+Tree。
这棵树的叶子节点通常不会存整行，而是存：

- `name`
- 对应记录的主键 `id`

这样通过 `name` 查到主键后，再回主键索引查整行。

15.9 为什么说索引是“有序结构”

索引之所以快，一个很重要的原因是它不是乱放的，而是按键值顺序组织的。

例如索引键是：

- 10
- 20
- 30
- 40

那么 B+Tree 会按这个顺序把它们分布到各个页中，并保持整体有序。

有序带来的好处：

- 可以快速做等值查询
- 可以快速做范围查询
- 可以辅助排序
- 可以辅助分组

这也是为什么数据库里高性能索引大多强调“有序性”。

15.10 建索引时字段顺序为什么重要

对联合索引来说，建立索引时字段的顺序本身就是索引设计的一部分。

例如：

```
create index idx_name_age_city on user(name, age, city);
```

它不是简单地“给三个字段都建索引”，而是把键组织成：

- 先按 `name`
- 再按 `age`
- 再按 `city`

所以索引里的排序规则是有先后顺序的。
这也就是最左前缀原则产生的根本原因。

15.11 索引的作用

- 加快查询速度
- 提高排序和分组效率
- 支持唯一性约束

15.12 常见索引类型

- 主键索引
- 唯一索引
- 普通索引
- 联合索引
- 全文索引

15.13 InnoDB 索引底层结构

InnoDB 索引底层通常使用 `B+Tree` 。

15.14 为什么用 B+Tree

可以这样回答：

- B+Tree 层数低，磁盘 IO 少
- 非叶子节点只存索引键，能容纳更多分支
- 叶子节点天然有序，适合范围查询
- 比二叉树、红黑树更适合数据库
- 比 Hash 更支持排序和范围查询

15.15 聚簇索引和二级索引

InnoDB 中：

- 主键索引是聚簇索引
- 二级索引叶子节点通常存储索引列和主键值

这就引出两个重要概念：

- 回表
- 覆盖索引

15.16 回表

先通过二级索引找到主键，再通过主键去聚簇索引查整行数据，叫回表。

15.17 覆盖索引

查询所需字段全部在索引中，可以直接返回结果，不需要回表，叫覆盖索引。

15.18 联合索引

```
create index idx_name_age_city on user(name, age, city);
```

符合最左前缀原则：

- 能用 `name`
- 能用 `name, age`
- 能用 `name, age, city`
- 不能直接跳过 `name` 只用 `age`

15.19 索引失效常见场景

- 对索引列使用函数
- 对索引列进行计算
- 隐式类型转换
- `like '%abc'`
- 联合索引不满足最左前缀
- 使用不合理的 `or`
- 数据区分度低，优化器不走索引

示例：

```
select * from user where left(name, 1) = 'A';
```

15.20 适合建索引的字段

- 高频查询字段
- 连接字段
- 排序字段
- 分组字段
- 唯一性要求高的字段

15.21 不适合建索引的字段

- 表很小
- 更新特别频繁
- 区分度很低，如性别
- 重复索引过多

15.22 InnoDB 的数据组织方式

InnoDB 是索引组织表，数据本身就是按照主键顺序存储在聚簇索引中的。

可以这样理解：

- 表数据并不是“单独放一份，索引再放一份”
- 对 InnoDB 来说，主键索引叶子节点里放的就是整行记录
- 所以 InnoDB 表一定有且必须依赖一个聚簇索引

如果表没有显式主键，InnoDB 会：

1. 优先选择第一个非空唯一索引作为聚簇索引
2. 如果没有，就生成隐藏主键

面试重点：

- 为什么建议 InnoDB 表显式设置主键

- 因为隐藏主键不可控，不利于维护和性能分析

15.23 InnoDB 页和 B+Tree 的关系

InnoDB 磁盘管理的基本单位是页，默认页大小常见为 `16KB` 。

可以这样理解 B+Tree：

- 一个节点通常对应一个数据页
- 非叶子节点页存放键值和子节点指针
- 叶子节点页存放真正的数据记录或主键值

这意味着：

- MySQL 查询一次，不是读一条记录，而是读一个页
- B+Tree 的目标之一就是让树尽量“矮胖”，减少磁盘 IO 次数

15.24 B+Tree 的结构特点

B+Tree 与 B-Tree 区别

可以直接这样回答：

- B+Tree 的非叶子节点只存键，不存完整数据
- B+Tree 的数据全部在叶子节点
- B+Tree 的叶子节点之间通常通过链表连接

所以 B+Tree 更适合数据库：

- 单个节点可容纳更多键
- 树更矮，IO 更少
- 范围查询更高效

为什么 B+Tree 适合范围查询

因为叶子节点天然有序，且叶子页之间通常相互连接。

例如查询：

```
select * from user where id between 100 and 200;
```

数据库定位到起始叶子页后，可以顺着叶子节点链表继续往后扫，不需要频繁回到上层节点。

15.25 B+Tree 和其他结构对比

和二叉搜索树比

- 二叉搜索树极端情况下会退化成链表
- 树高太高，磁盘 IO 太多

和红黑树比

- 红黑树虽然平衡，但一个节点只存一个键
- 节点分支少，树高仍然较高
- 不适合磁盘型数据库

和 Hash 比

- Hash 查询等值很快
- 但 Hash 不支持范围查询
- 不支持排序
- 不利于 `order by`、`between`、`>`、`<` 这类场景

15.26 B+Tree 查找过程示意

假设有主键索引：

```
select * from user where id = 120;
```

查找过程大致是：

1. 从根节点页开始
2. 比较键值，决定进入哪个子页
3. 一层层向下查到叶子页
4. 在叶子页中定位到对应记录

如果树高为 3，那么大致只需要 3 次页访问就能定位到数据。

面试高频说法：

- B+Tree 的查询时间复杂度可以理解为 `O(logN)`
- 但数据库更关注的是页读取次数，而不是纯算法课里的比较次数

15.27 主键为什么建议自增

面试里经常会问“为什么 InnoDB 推荐使用自增主键”。

核心原因：

- 聚簇索引按主键顺序组织
- 自增主键插入通常追加到页尾
- 减少页分裂和页移动
- 插入局部性更好

如果主键是随机 UUID：

- 插入位置分散

- 容易引发页分裂
- 索引维护成本更高
- 缓存命中率可能更差

15.28 页分裂和页合并

页分裂

当一个页放满后，又要向该页中间插入记录，就可能发生页分裂。

影响：

- 需要申请新页
- 迁移部分数据
- 调整父节点指针
- 会带来额外 IO 和写放大

页合并

删除大量数据后，页空间利用率很低时，可能发生页合并。

面试重点：

- 自增主键的一个重要优势，就是减少随机插入导致的页分裂

15.29 二级索引为什么会回表

以表 `user(id, name, age, city)` 为例：

如果建立索引：

```
create index idx_name on user(name);
```

执行：

```
select * from user where name = 'Alice';
```

过程是：

1. 先在二级索引 `idx_name` 中找到 `name = 'Alice'`
2. 叶子节点拿到对应主键 `id`
3. 再回到主键索引中找整行记录

这就是回表。

如果查询改成：

```
select id, name from user where name = 'Alice';
```

因为二级索引叶子节点通常已经包含索引列和主键值，所以这时可能直接形成覆盖索引，避免回表。

15.30 联合索引案例

假设有索引：

```
create index idx_status_create_time on orders(status,
create_time);
```

下面几个查询的区别：

```
select * from orders where status = 1;
```

可以利用联合索引最左列。

```
select * from orders where status = 1 and create_time > '2025-
01-01';
```

可以同时利用联合索引。

```
select * from orders where create_time > '2025-01-01';
```

通常无法很好利用该联合索引，因为跳过了最左列 `status` 。

```
select * from orders where status = 1 order by create_time
desc;
```

如果索引设计合理，还可能同时帮助过滤和排序。

---

16. 事务

16.1 什么是事务

事务是一组 SQL 操作，要么全部成功，要么全部失败。

示例：

```
start transaction;

update account set money = money - 100 where id = 1;
update account set money = money + 100 where id = 2;

commit;
```

失败时：

```
rollback;
```

16.2 事务的四大特性 ACID

- `Atomicity`：原子性
- `Consistency`：一致性
- `Isolation`：隔离性
- `Durability`：持久性

16.3 ACID 解释

原子性：

- 事务不可分割，要么全部执行，要么全部不执行

一致性：

- 事务执行前后，数据必须保持业务规则正确

隔离性：

- 多个事务之间彼此不互相干扰

持久性：

- 事务提交后，结果永久保存

---

17. 并发问题与隔离级别

17.1 并发问题

脏读：

- 一个事务读到了另一个事务未提交的数据

不可重复读：

- 同一个事务中，两次读取同一条记录，结果不同

幻读：

- 同一个事务中，两次按条件读取，第二次多出或少了几行

17.2 MySQL 四种隔离级别

- `read uncommitted`
- `read committed`
- `repeatable read`
- `serializable`

17.3 隔离级别与问题对应

`read uncommitted`：

- 可能出现脏读、不可重复读、幻读

`read committed`：

- 解决脏读
- 仍可能不可重复读、幻读

`repeatable read`：

- 解决脏读和不可重复读
- MySQL 中配合 MVCC 和锁机制处理幻读问题

`serializable`：

- 隔离级别最高
- 并发性能最差

17.4 默认隔离级别

MySQL InnoDB 默认隔离级别是：

- `repeatable read`

---

18. 锁

18.1 常见锁

- 表锁
- 行锁
- 共享锁
- 排他锁
- 意向锁
- 间隙锁
- 临键锁

18.2 共享锁和排他锁

共享锁：

- 又叫读锁
- 多个事务可同时持有共享锁

排他锁：

- 又叫写锁
- 持有排他锁时，其他事务不能再加共享锁或排他锁

18.3 行锁特点

- InnoDB 主要使用行锁
- 行锁是基于索引实现的
- 如果没走索引，可能锁的范围会扩大

18.4 锁定读

```
select * from user where id = 1 for update;
select * from user where id = 1 for share;
```

18.5 间隙锁和临键锁

间隙锁

间隙锁锁的不是某一条记录，而是某个索引区间之间的“间隙”。

它主要用于防止幻读。

例如：

- 已有索引值 `10, 20, 30`
- 如果事务锁住了 `(10, 20)` 这个间隙
- 其他事务就不能插入 `11~19` 之间的新值

临键锁

临键锁可以理解为：

- 记录锁 + 间隙锁

它锁住当前记录以及它前面的间隙，是 InnoDB 在可重复读级别下处理范围查询时非常重要的锁机制。

18.6 为什么行锁是“加在索引上”

这句话面试里经常会听到。

更准确地说：

- InnoDB 是通过索引项来定位记录并加锁的
- 如果 SQL 不能利用索引准确定位范围，锁范围可能会扩大

例如：

```
update user set age = 30 where id = 100;
```

如果 `id` 是主键，那么锁定范围很精确。

但如果执行：

```
update user set age = 30 where name = 'Alice';
```

如果 `name` 上没有索引，MySQL 可能需要扫描大量记录，锁影响范围也可能变大。

---

19. MVCC

MVCC，Multi-Version Concurrency Control，多版本并发控制。

19.1 作用

- 提高并发性能
- 普通读操作尽量不加锁
- 在一致性和性能之间取得平衡

19.2 核心思想

- 一条记录可能存在多个版本
- 旧版本通过 `undo log` 保存
- 事务通过 `Read View` 判断自己可见哪个版本

19.3 面试常用答法

InnoDB 在 `read committed` 和 `repeatable read` 等隔离级别下，会通过 MVCC 实现快照读。普通 `select` 通常不加锁，而 `update`、`delete`、`select for update` 等属于当前读，通常需要加锁。

19.4 快照读和当前读

快照读

普通 `select` 大多属于快照读。

特点：

- 读取的是一致性视图
- 通常不加锁
- 借助 MVCC 实现

例如：

```
select * from user where id = 1;
```

当前读

读取最新版本数据，并且通常需要加锁。

常见语句：

- `select ... for update`
- `select ... for share`
- `update`
- `delete`
- `insert`

19.5 undo log、redo log、binlog 简要区分

这是面试里非常容易连在一起问的一组概念。

undo log

作用：

- 保存旧版本数据
- 支持事务回滚
- 支持 MVCC

redo log

作用：

- 记录对页的物理修改
- 保证已提交事务的持久性
- 崩溃恢复时可重放

binlog

作用：

- MySQL Server 层的逻辑日志
- 常用于主从复制和数据恢复

面试常见一句话总结：

- `undo log` 用于回滚和 MVCC
- `redo log` 用于崩溃恢复
- `binlog` 用于复制和归档

---

20. EXPLAIN 执行计划

20.1 explain 的作用

用于查看 SQL 如何执行，比如：

- 是否走索引
- 走了哪个索引
- 扫描多少行
- 是否用了临时表
- 是否发生文件排序

20.2 示例

```
explain select * from emp where dept_id = 1;
```

20.3 重点关注字段

- `id`
- `select_type`
- `table`
- `type`
- `possible_keys`
- `key`
- `rows`
- `extra`

20.4 type 常见等级

一般可粗略理解为性能从好到差：

- `system`
- `const`
- `eq_ref`
- `ref`
- `range`
- `index`
- `all`

面试重点：

- `all` 往往是全表扫描
- `range` 通常还可以
- `ref`、`eq_ref` 一般较优

20.5 extra 常见关键词

- `Using index`： 可能使用了覆盖索引
- `Using where`： 做了条件过滤
- `Using temporary`： 可能用了临时表
- `Using filesort`： 可能做了额外排序

---

21. SQL 优化思路

21.1 基础优化原则

1. 先看 `explain`
2. 给高频过滤、关联、排序字段建立合适索引
3. 避免 `select *`
4. 尽量走覆盖索引
5. 避免在索引列上做函数或运算
6. 联合索引注意最左前缀
7. 控制返回行数
8. 避免深分页
9. 避免大事务
10. 批量插入优于单条循环插入

21.2 深分页问题

```
select * from user limit 100000, 10;
```

问题：

- MySQL 可能需要先扫描并跳过前 100000 行，代价很高

常见优化思路：

- 基于主键或索引字段做范围分页

例如：

```
select * from user
where id > 100000
order by id
limit 10;
```

21.3 小表驱动大表

在多表关联时，通常希望让结果集更小、过滤性更强的表先参与驱动。

原因：

- 可以减少后续关联次数
- 减少中间结果集规模

当然，实际执行顺序是否如此，还要看优化器和执行计划。

21.4 order by / group by 优化思路

如果 `where + order by` 或 `where + group by` 能和索引顺序匹配，性能通常更好。

例如：

```
create index idx_status_create_time on orders(status,
create_time);
```

查询：

```
select id, status, create_time
from orders
where status = 1
order by create_time desc
limit 20;
```

如果命中联合索引，往往能同时减少过滤、排序和回表成本。

---

22. SQL 实际案例大全

这一章补充更多真实业务中常见的 SQL 场景，面试时很容易被问到。

22.1 用户表基础查询

表结构假设：

- `user(id, name, age, city, register_time, status)`

查询年龄大于 25 且状态正常的用户：

```
select id, name, age
from user
where age > 25
  and status = 1;
```

查询北京用户，按注册时间倒序：

```
select id, name, city, register_time
from user
where city = 'Beijing'
order by register_time desc;
```

22.2 条件统计案例

统计每个城市的用户数：

```
select city, count(*) as user_cnt
from user
group by city;
```

统计每个城市状态正常的用户数：

```
select city, count(*) as normal_user_cnt
from user
where status = 1
group by city;
```

统计用户总数、平均年龄、最大年龄：

```sql
select count(*) as total_cnt,
       avg(age) as avg_age,
       max(age) as max_age
from user;
```

22.3 订单表查询案例

表结构假设：

- `orders(id, user_id, amount, status, create_time)`

查询近 30 天订单：

```sql
select *
from orders
where create_time >= date_sub(now(), interval 30 day);
```

查询支付成功的订单总金额：

```sql
select sum(amount) as total_amount
from orders
where status = 'paid';
```

查询每个用户的下单次数和总消费：

```sql
select user_id,
       count(*) as order_cnt,
       sum(amount) as total_amount
from orders
group by user_id;
```

22.4 多表 join 业务案例

表结构假设：

- `user(id, name, city)`
- `orders(id, user_id, amount, status, create_time)`

查询每个订单对应的用户名：

```
select o.id, o.amount, o.status, u.name
from orders o
join user u on o.user_id = u.id;
```

查询每个用户的总订单数，没有下单的用户也要显示：

```
select u.id, u.name, count(o.id) as order_cnt
from user u
left join orders o on u.id = o.user_id
group by u.id, u.name;
```

查询没有下单的用户：

```
select u.*
from user u
left join orders o on u.id = o.user_id
where o.id is null;
```

22.5 TopN 查询案例

查询工资最高的前 5 个员工：

```
select *
from emp
order by salary desc
limit 5;
```

查询每个部门工资前 2 名：

```
select *
from (
    select e.*,
           row_number() over(partition by dept_id order by
salary desc) as rn
    from emp e
) t
where rn <= 2;
```

22.6 去重和重复数据处理

查询重复手机号：

```
select phone, count(*) as cnt
from user
group by phone
having count(*) > 1;
```

删除重复手机号，只保留最小 id：

```
delete from user
where id not in (
    select keep_id
    from (
        select min(id) as keep_id
        from user
        group by phone
    ) t
);
```

22.7 连续登录案例

表结构假设：

- `user_login(user_id, login_date)`

查询连续登录 3 天及以上的用户，思路通常是用窗口函数构造分组键：

```
select user_id
from (
    select user_id,
           login_date,
           date_sub(login_date, interval row_number()
over(partition by user_id order by login_date) day) as grp
    from user_login
) t
group by user_id, grp
having count(*) >= 3;
```

这是非常经典的连续问题题型。

22.8 留存和活跃案例

查询每天登录用户数：

```
select login_date, count(distinct user_id) as dau
from user_login
group by login_date;
```

查询次日留存用户数，思路是把某天登录的用户和第二天登录的用户做自连接：

```
select a.login_date,
       count(distinct a.user_id) as retained_cnt
from user_login a
join user_login b
  on a.user_id = b.user_id
 and b.login_date = date_add(a.login_date, interval 1 day)
group by a.login_date;
```

22.9 漏斗统计案例

表结构假设：

- `event_log(user_id, event_name, event_time)`

统计完成"浏览 -> 加购 -> 支付"的用户数：

```sql
select count(*) as convert_user_cnt
from (
    select user_id,
           max(case when event_name = 'view' then 1 else 0 end)
as viewed,
           max(case when event_name = 'cart' then 1 else 0 end)
as carted,
           max(case when event_name = 'pay' then 1 else 0 end)
as paid
    from event_log
    group by user_id
) t
where viewed = 1 and carted = 1 and paid = 1;
```

22.10 分页案例

普通分页：

```sql
select *
from orders
order by id
limit 0, 20;
```

深分页优化：

```sql
select *
from orders
where id > 100000
order by id
limit 20;
```

22.11 排名与占比案例

查询每个部门员工工资排名：

```sql
select emp_id,
       dept_id,
       salary,
       dense_rank() over(partition by dept_id order by salary
desc) as rk
from emp;
```

查询每个用户消费金额以及其占全部消费的比例：

```sql
select user_id,
       sum(amount) as total_amount,
       sum(amount) / sum(sum(amount)) over() as ratio
from orders
group by user_id;
```

---

23. 基础 SQL 模板

22.1 查询每个部门人数

```sql
select dept_id, count(*) as cnt
from emp
group by dept_id;
```

22.2 查询每个部门平均工资

```sql
select dept_id, avg(salary) as avg_salary
from emp
group by dept_id;
```

22.3 查询平均工资大于 10000 的部门

```sql
select dept_id, avg(salary) as avg_salary
from emp
group by dept_id
having avg(salary) > 10000;
```

22.4 查询工资高于平均工资的员工

```
select *
from emp
where salary > (select avg(salary) from emp);
```

22.5 查询每个部门工资最高的人

```
select e.*
from emp e
join (
    select dept_id, max(salary) as max_salary
    from emp
    group by dept_id
) t on e.dept_id = t.dept_id and e.salary = t.max_salary;
```

22.6 查询每个部门工资前 3 名

```
select *
from (
    select e.*,
           row_number() over(partition by dept_id order by
salary desc) as rn
    from emp e
) t
where rn <= 3;
```

22.7 查询重复数据

```
select email, count(*) as cnt
from user
group by email
having count(*) > 1;
```

22.8 删除重复数据，保留最小 id

```
delete from user
where id not in (
    select keep_id
    from (
        select min(id) as keep_id
        from user
        group by email
    ) t
);
```

22.9 查询没有下单的用户

```
select u.*
from user u
left join orders o on u.id = o.user_id
where o.user_id is null;
```

22.10 查询第二高工资

```
select max(salary)
from emp
where salary < (select max(salary) from emp);
```

或者：

```
select distinct salary
from emp
order by salary desc
limit 1, 1;
```

22.11 按月统计销售额

```
select date_format(order_time, '%Y-%m') as mon,
       sum(amount) as total_amount
from orders
group by date_format(order_time, '%Y-%m');
```

24. 高频问答版

23.1 什么是 MySQL

MySQL 是关系型数据库管理系统，使用表来组织和存储数据，支持 SQL 查询、事务、索引、并发控制等能力。

23.2 SQL 的分类有哪些

SQL 可以分为 DDL、DML、DQL、DCL、TCL。

- DDL：定义数据库对象
- DML：操作数据
- DQL：查询数据
- DCL：权限控制
- TCL：事务控制

23.3 `char` 和 `varchar` 有什么区别

`char` 是定长字符串，长度固定；`varchar` 是变长字符串，更节省空间。实际业务中 `varchar` 更常用。

23.4 `primary key` 和 `unique` 的区别

主键一定唯一且不能为空，一个表只能有一个主键；唯一约束也要求唯一，但一个表可以有多个唯一约束，通常允许 `null`。

23.5 `delete`、`truncate`、`drop` 的区别

`delete` 删除数据，可带条件；`truncate` 清空整张表，效率通常更高；`drop` 删除表结构和数据。

23.6 `where` 和 `having` 的区别

`where` 用于分组前过滤，`having` 用于分组后过滤，通常与聚合函数配合。

23.7 `count(*)`、`count(1)`、`count(字段)` 的区别

`count(*)` 与 `count(1)` 通常都统计总行数，`count(字段)` 只统计该字段不为 `null` 的记录数。

23.8 `inner join` 和 `left join` 的区别

`inner join` 只返回匹配成功的数据；`left join` 会保留左表全部数据，右表匹配不到则补 `null`。

23.9 SQL 的执行顺序是什么

逻辑顺序通常是：

`from -> join -> on -> where -> group by -> having -> select -> order by -> limit`

23.10 什么是索引，有什么作用

索引是加速查询的数据结构，作用是提高查询、排序、分组和关联的效率。

23.11 为什么 MySQL 索引常用 B+Tree

因为 B+Tree 高度低、磁盘 IO 少、适合范围查询和排序，更适合数据库场景。

23.12 什么是聚簇索引

聚簇索引是指叶子节点直接存储整行数据。InnoDB 的主键索引就是聚簇索引。

23.13 什么是回表

通过二级索引找到主键后，再到主键索引查整行数据的过程叫回表。

23.14 什么是覆盖索引

查询需要的列都在索引中，不需要回表，这叫覆盖索引。

23.15 什么是最左前缀原则

联合索引从最左边字段开始连续匹配，才能较好地使用索引，这叫最左前缀原则。

23.16 索引什么时候会失效

常见情况包括：

- 对索引列使用函数
- 对索引列做运算
- 隐式类型转换
- `like '%xx'`
- 联合索引未满足最左前缀

23.17 什么是事务

事务是一组操作，要么全部成功，要么全部失败，用于保证数据一致性。

23.18 事务的 ACID 是什么

- 原子性
- 一致性

- 隔离性
- 持久性

23.19 什么是脏读、不可重复读、幻读

脏读是读到未提交数据，不可重复读是同一行两次读取结果不同，幻读是同一条件两次查询结果集行数不同。

23.20 MySQL 的隔离级别有哪些，默认是什么

有四种：

- `read uncommitted`
- `read committed`
- `repeatable read`
- `serializable`

InnoDB 默认是 `repeatable read` 。

23.21 什么是 MVCC

MVCC 是多版本并发控制，通过维护数据多个版本，让普通读在很多情况下不加锁，从而提高并发性能。

23.22 普通 `select` 为什么通常不加锁

因为 InnoDB 会通过 MVCC 实现快照读；而 `update` 、 `delete` 、 `select for update` 等属于当前读，通常会加锁。

23.23 行锁和表锁区别

表锁锁整张表，并发差；行锁锁部分记录，并发高。InnoDB 主要使用行锁。

23.24 什么是 explain

`explain` 用来分析 SQL 执行计划，看是否走索引、扫描多少行、是否有临时表、排序等。

23.25 SQL 优化一般怎么做

先看执行计划，再从索引、SQL 写法、返回字段、分页方式、事务大小等方面做优化。

---

25. 面试中的经典答题模板

24.1 定义类问题答法

适用于：

- 什么是索引
- 什么是事务
- 什么是 MVCC
- 什么是覆盖索引

答题结构：

1. 先说定义
2. 再说原理
3. 再说优点
4. 再说适用场景
5. 最后补常见坑

例如“什么是覆盖索引”：

覆盖索引是指查询所需字段都能直接从索引中获取，不需要回表。它的好处是减少 IO，提高查询效率，常用于高频查询场景。但索引不是越多越好，维护索引也会带来写入成本。

24.2 区分类问题答法

适用于：

- `where` 和 `having`
- `inner join` 和 `left join`
- `char` 和 `varchar`
- `delete` 和 `truncate`

答题结构：

1. 先说核心区别
2. 再说使用场景
3. 最后补一个常见例子

24.3 原理类问题答法

适用于：

- 为什么索引用 B+Tree
- 为什么会回表
- 为什么会有幻读

答题结构：

1. 先回答结论
2. 再解释底层机制
3. 最后讲业务影响

---

26. 面试速记

25.1 20 句高频速记

1. SQL 是结构化查询语言，MySQL 是关系型数据库。
2. SQL 分为 DDL、DML、DQL、DCL、TCL。
3. 金额字段优先使用 `decimal` 。
4. `varchar` 比 `char` 更常用。
5. SQL 的执行顺序不是书写顺序。
6. `where` 是分组前过滤， `having` 是分组后过滤。
7. `count(字段)` 不统计 `null` 。
8. `left join` 会保留左表全部记录。
9. InnoDB 是 MySQL 常用存储引擎。
10. InnoDB 默认隔离级别是 `repeatable read` 。
11. 事务具有 ACID 四大特性。
12. 并发问题有脏读、不可重复读、幻读。
13. InnoDB 索引底层一般是 B+Tree。
14. 主键索引是聚簇索引。
15. 二级索引查询整行数据时可能会回表。
16. 覆盖索引可以减少回表。
17. 联合索引遵守最左前缀原则。
18. 对索引列做函数操作可能导致索引失效。
19. SQL 调优第一步是看 `explain` 。
20. 深分页性能差，尽量使用基于索引的范围分页。

25.2 面试优先复习顺序

1. `select`、`where`、`group by`、`having`
2. `join`、子查询、聚合函数
3. 索引、最左前缀、回表、覆盖索引
4. 事务、隔离级别、锁、MVCC
5. `explain` 和 SQL 优化
6. 窗口函数和经典题型

---

27. 学习建议

如果你现在是面试前突击，建议这样用这份文档：

第一遍：

- 把 1 到 14 章快速过一遍，重建基础概念

第二遍：

- 重点看 15 到 23 章，把索引、事务、锁、MVCC、问答版背下来

第三遍：

- 重点刷第 22 章 SQL 模板和第 25 章速记

如果面试官偏实战，你至少要做到：

- 能写基础 `select`
- 能写 `group by`
- 能写 `join`
- 能讲清索引

- 能讲清事务和隔离级别
- 能说出 `explain` 的用途

---

28. 官方参考

- MySQL Reference Manual: https://dev.mysql.com/doc/refman/en/
- SELECT Statement: https://dev.mysql.com/doc/refman/8.4/en/select.html
- JOIN Clause: https://dev.mysql.com/doc/refman/8.4/en/join.html
- Aggregate Functions: https://dev.mysql.com/doc/refman/en/aggregate-functions-and-modifiers.html
- Window Functions: https://dev.mysql.com/doc/refman/en/window-functions.html
- InnoDB Transaction Model: https://dev.mysql.com/doc/mysql/en/innodb-transaction-model.html
- InnoDB Locking Reads: https://dev.mysql.com/doc/refman/8.3/en/innodb-locking-reads.html
- EXPLAIN Statement: https://dev.mysql.com/doc/refman/8.4/en/explain.html